

A RESEARCH AND IMPLEMENTATION PROPOSAL

Forge: Finite Certificates for Infinite Behaviors

Observable closure, target-generated polynomial ideals,
and boundary-safe symbolic summation

Three executable extensions beyond the current design

Prepared for Vladimir Reshetnikov

15 September 2026

What accompanies this article. Exact Python searches, independently implemented standard-library checkers, 38 proof-certificate bundles, 17 concrete counterexamples, a 59-instance experimental corpus, mutation records, and Lean source specimens. The 117 regression tests pass.

Evidence boundary. The Python prototypes were executed. The Lean source was not compiled: no Lean executable was available. This is not an installed **forge** tactic, a kernel-verification result, or a measured improvement over **grind**.

Novelty boundary. The contribution is an incremental design and tested implementation for Forge, not a claim to have invented weighted-automaton equivalence, polynomial-ideal analysis, or creative telescoping.

Abstract

The present Forge design already contains a substantial proof-planning layer, induction and lemma discovery, polynomial recurrence synthesis, arithmetic witness construction, and several exact certificate families. Recommending another scheduler or another generic invariant template would not advance it. This article develops three narrower but substantially stronger mechanisms. First, observable-space closure decides a finite-dimensional rational linear-behavior fragment and returns either small matrix certificates or an explicit distinguishing input word. Second, target-generated pullback ideals construct mutually supporting polynomial invariants from the goal itself, avoiding the bilinear search caused by simultaneously guessing invariants and multipliers. An ascending-ideal argument establishes termination and completeness for the stated unguarded polynomial fragment, while practical budgets remain explicit. Third, a pole-free binomial-sum worker synthesizes recurrence and syzygy certificates by exact linear algebra, preserving endpoints that rational Gosper certificates can obscure. We give soundness proofs, failure examples, Lean and Leant integration contracts, concrete library choices, a staged implementation plan, and reproducible experiments. The recorded run has 38 certificate successes, 17 replayed counterexamples, and four bounded unknowns; 818 single-entry mutations are rejected. None of these counts is a Lean tactic success rate. The central proposal is to add mathematical closure mechanisms whose output explains an entire infinite family of executions using finitely many identities.

Contents

1	The incremental objective	2
2	Worker A: target-directed observable-space closure	4
3	Worker B: target-generated pullback ideals	6
4	Worker C: pole-free symbolic summation certificates	11
5	Integrating the workers into Forge and Leant	15
6	Implementation and experimental evidence	19
7	A concrete implementation sequence	21
8	Conclusion	24
A	Certificate schemas and executable entry points	25
B	Reproduction and archive map	26
C	Research questions left by the implementation	27
	References	27

1 The incremental objective

The most useful next contribution to Forge is not more orchestration. It is a small number of new reasoning domains with a precise admission test, a concrete algorithm, and a short route from search output to a Lean proof. This proposal adds three such domains. Their common feature is that they replace unbounded execution or unfolding by a finite closure computation.

1.1 What the audit establishes

The Forge snapshot reviewed here is 674521027d968d59f7b83220ed52304a85cb55e2. Its merged article, status ledger, structural algorithms, and provenance appendix already cover a great deal: demand-directed theorem use, structural induction and generalization, auxiliary lemma synthesis, exact polynomial sums, state-machine conserved quantities, polynomial equality multipliers, and nonlinear and Boolean certificate workers [1, 2, 3, 4]. Those ideas are prerequisites, not contributions of this article.

The important boundary is more specific. The implemented state-machine certificate in the structural section checks

$$I(F(s)) = I(s),$$

whereas an invariant only needs its zero set to be preserved. The same section correctly notes that simultaneously searching for unknown I_j and unknown H_{ij} in

$$I_i(F(s)) = \sum_j H_{ij}(s) I_j(s)$$

is a bilinear problem. The present proposal supplies a concrete alternative: generate the required invariant ideal from the target, then compute the multipliers after the generating set has stabilized. This changes both the search problem and the fragment for which a completeness argument is available.

The audit read the merged algorithm and status material, rather than claiming a byte-for-byte review of all nine historical submissions. Some long connector responses were truncated; the successive structural ranges and the provenance inventory were used to resolve the relevant boundary. An unresponsive repository search index was not used as evidence that a term was absent. The exact locations and limitations are recorded in `docs/audit.md`.

Area	Existing ingredient	Increment supplied here
Recursive arithmetic	Polynomial differences and conserved quantities	Exact closure of all target observations under switched linear updates.
Polynomial invariants	General inductiveness suggested; multiplier identity format known	Target-generated ascending ideals; automatic strengthening; concrete counterexample extraction.
Finite sums	Additive polynomial recurrences	Binomial powers and moments; a pole-free telescoping template and exact coefficient synthesis.
Leant behavior	Exact candidate checks by <code>decide</code> and bounded <code>simp</code>	A restricted universal-behavior adapter based on checked fold compilation and closure certificates.

Table 1: The proposed delta, not a restatement of the existing architecture.

1.2 A shared question, three different closures

Consider a state s , an update F , and an observation q . Instead of asking “which invariant should I guess?”, ask:

What additional observations must vanish now to ensure that the desired observation vanishes after every possible future update?

For linear maps, observations are row vectors and closure is linear span. For polynomial maps, observations are polynomials and closure is an ideal. For symbolic sums, observations are shifted summands and closure is taken modulo known identities and telescoping differences. The mathematical structures differ, but each gives a disciplined way to manufacture the auxiliary facts needed by an induction proof.

This is more useful than a collection of isolated examples. A single linear worker covers equivalence of finite-state numeric folds, linear recurrence implementations, and affine accumulator programs. A single ideal worker covers coupled nonlinear recurrences and invariants whose values change but whose zero sets do not. A single summation worker produces proofs of recurrences for whole families of finite sums rather than evaluating isolated inputs.

1.3 The notation and outcome contract

All executed searches use exact rational arithmetic. A finite alphabet Σ indexes finitely many total updates. The execution of a word $w = a_1 \cdots a_t$ is

$$F_w(s) = F_{a_t}(\cdots F_{a_2}(F_{a_1}(s)) \cdots), \quad F_\epsilon(s) = s.$$

This convention is important: backward multiplication of observations reverses how a new symbol is attached to a recorded forward execution word.

A successful result is a certificate for the exact supplied problem. A negative result is a concrete counterexample evaluated in that problem. Resource exhaustion or failure of a bounded template gives UNKNOWN. In the summation worker, success certifies a *recurrence*, not automatically a closed form or uniqueness of its solution. These distinctions determine what each result is allowed to do; they are not merely labels in a report.

The article distinguishes mathematical theorems about idealized algorithms, executed Python validation, uncompiled Lean source, and proposed integration. In particular, the proof that an unbounded ideal algorithm terminates does not turn an interrupted Gröbner-basis computation into a negative answer.

1.4 Relationship to established work

Observable closure belongs to the linear-algebraic theory of weighted-automaton equivalence; Kiefer gives a compact treatment of the classical field-based algorithms [9]. Polynomial backward reasoning by ideals has direct antecedents in the program-analysis work of Cachera, Jensen, Jobin, and Kirchner [10, 11]. Creative telescoping is an established symbolic summation method, including modern algorithms that keep certificates compact [12]. The specific contribution here is to turn these ideas into deliberately small, independently replayable workers at Forge’s existing boundary, and to expose the semantic obligations their integration actually needs.

No claim is made that the demonstrations would defeat every existing tactic or library lemma. A one-line application of a previously proved theorem may beat any search. The improvement sought is the ability to discover a finite explanation when that theorem or invariant has not yet been supplied.

2 Worker A: target-directed observable-space closure

2.1 The exact fragment

Let $A_a \in \mathbb{Q}^{D \times D}$ for each $a \in \Sigma$, let $s_0 \in \mathbb{Q}^D$, and let $q \in \mathbb{Q}^{1 \times D}$. The problem is

$$\forall w \in \Sigma^*, \quad qA_w s_0 = 0, \quad A_{a_1 \dots a_t} = A_{a_t} \dots A_{a_1}. \quad (1)$$

All words are permitted. A constrained language of inputs needs either a product with an explicitly supplied finite automaton or a separate proof that the input restriction is respected. It must not be silently assumed.

Affine updates are included by homogenization:

$$s' = M_a s + b_a \quad \rightsquigarrow \quad \begin{bmatrix} s' \\ 1 \end{bmatrix} = \begin{bmatrix} M_a & b_a \\ 0 & 1 \end{bmatrix} \begin{bmatrix} s \\ 1 \end{bmatrix}.$$

An affine target is homogenized in the same way. Rational constants remain rational; no floating-point rank test is needed or appropriate.

For equivalence of two implementations, form their product state and block updates:

$$A_a = \begin{bmatrix} A_a^{(1)} & 0 \\ 0 & A_a^{(2)} \end{bmatrix}, \quad q = \begin{bmatrix} o_1 & -o_2 \end{bmatrix}.$$

Initial outputs and every subsequent output are then compared by a single zero-observation goal. Equal output does not require equal internal states, equal state dimensions, or the same coordinate system.

2.2 The algorithm

Start from the target row. Keep a basis G of the space generated by the rows seen so far. Whenever an independent row r is admitted, enqueue rA_a for each action. Dependent rows need not be expanded because multiplying their known linear combination by A_a produces the corresponding combination of already scheduled successor rows.

Each queued row carries a concrete forward word. A row representing qA_w becomes $qA_w A_a$ after right multiplication. Under our execution convention that represents the word aw , not wa .

```

queue := [(q, empty_word)]
basis := []
while queue is not empty:
    (r, w) := pop_front(queue)
    if r * initial != 0:
        replay w in the original matrices
        return Counterexample(w)
    if r lies in span(basis):
        continue
    append r to basis
    for action a:
        push_back(queue, (r * A[a], prepend(a, w)))
return exact closure coefficients for the completed basis

```

The test of rs_0 occurs before discarding a dependent row. It is harmless to instead perform it after a sound dependence test and propagate the corresponding zero-value argument; the explicit test makes the counterexample route easier to audit. The search implementation uses SymPy rational matrices. The checker uses separately written loops over Python Fraction, and imports none of the search code.

2.3 A certificate without rank computation

The search returns an $r \times D$ matrix G , a row $t \in \mathbb{Q}^{1 \times r}$, and matrices $C_a \in \mathbb{Q}^{r \times r}$ satisfying

$$q = tG, \tag{2}$$

$$GA_a = C_a G \quad (a \in \Sigma), \tag{3}$$

$$Gs_0 = 0. \tag{4}$$

The checker verifies these finite identities. It does not check that the rows of G are independent, that they were obtained by a particular search, or that r is minimal. Those conditions explain the algorithm's efficiency, but they are unnecessary for soundness.

Theorem 2.1 (Observable certificate soundness). *If (2)–(4) hold, then (1) holds.*

Proof. The invariant is $Gs = 0$. If it holds before action a , then $G(A_a s) = (GA_a)s = (C_a G)s = C_a(Gs) = 0$. It holds initially by (4), hence after every finite word by induction. Finally, $qs = tGs = 0$ by (2). $\square$

This is an inductive invariant, not a numerical approximation to a reachable set. The certificate can contain redundant rows and still be valid. Conversely, a claimed invariant basis without (2) might prove something irrelevant: the target-membership equation is indispensable.

2.4 Termination, completeness, and a sharp witness bound

Theorem 2.2 (Finite-dimensional coverage). *Without operational cutoffs, observable closure terminates and decides (1). If the goal is false and $q \neq 0$, a distinguishing word of length at most $D - 1$ exists.*

Proof. At most D independent rows can be admitted. Each admitted row creates $|\Sigma|$ successors, so the queue has at most $1 + |\Sigma|D$ entries over the entire run. On normal termination every basis row is closed under right multiplication by every action; induction therefore places every qA_w in the final row space. If all its basis rows annihilate s_0 , all observations do; otherwise a tested row supplies a concrete word.

For the length bound let V_i be the span of all target rows arising from words of length at most i . If $V_i = V_{i+1}$, the space is closed under each action and the chain has stabilized. Starting with the one-dimensional space generated by nonzero q , at most $D - 1$ strict dimension increases are possible. Thus V_{D-1} contains every target row. If its evaluation on s_0 is not identically zero, at least one of its generating words of length at most $D - 1$ has nonzero evaluation. $\square$

The supplied delayed-error fixtures attain this bound. Take a shift matrix with ones just above the diagonal, initial state e_D , and target e_1^T . Every output before step $D - 1$ is zero; the next is one. Testing only the first several inputs would mistake these systems for correct ones. The finite-dimensional argument, rather than the number of sampled inputs, tells us when a conclusion is justified.

With a maintained echelon basis, row multiplication and membership reduction cost $O(D^2)$ field operations per candidate; the search therefore admits an $O(|\Sigma|D^3)$ implementation. Bit complexity additionally depends on exact coefficient growth. The prototype recomputes ranks using SymPy rather than maintaining the optimized incremental representation, so the cubic bound is a production design target, not a claimed bound for every internal operation of the delivered Python implementation. This distinction agrees with the algorithmic refinements in the weighted-automata literature [9].

2.5 A small example that conservation misses

For input $a \in \{0, 1\}$, update

$$x' = 2x + a, \quad y' = 2y + 2a, \quad (x_0, y_0) = (0, 0).$$

The desired observation is $q = 2x - y$. Its value changes by the rule $q' = 2q$, not $q' = q$. It is nevertheless zero after every input word. Homogenizing gives

$$A_a = \begin{bmatrix} 2 & 0 & a \\ 0 & 2 & 2a \\ 0 & 0 & 1 \end{bmatrix}, \quad G = \begin{bmatrix} 2 & -1 & 0 \end{bmatrix}, \quad C_a = [2].$$

The certificate has one invariant row, regardless of word length. Neither coefficient interpolation over the iteration count nor a guessed closed form for each state coordinate is necessary.

The corpus also checks product systems related by an invertible triangular change of coordinates. They are useful because a structural comparison of the two state vectors fails, whereas observable closure automatically discovers precisely the linear combinations needed to establish output equivalence.

2.6 Beyond a finite alphabet: a careful extension

Suppose an update depends polynomially on a fresh scalar input u :

$$A(u) = \sum_{j=0}^d u^j M_j.$$

A sufficient universal certificate consists of

$$GM_j = C_j G \quad (0 \leq j \leq d),$$

because then $GA(u) = (\sum_j u^j C_j)G$ for every u . Over an infinite field, vanishing of an output polynomial for all choices of inputs forces all its coefficients to vanish, so coefficient closure gives a completeness route for this precise polynomial-input linear-state fragment.

A coefficient matrix is not an input symbol, however. If a coefficient row does not annihilate the initial state, its index sequence is only evidence of a nonzero output polynomial. To produce an actual counterexample, instantiate that polynomial on an exact grid whose coordinate sizes exceed the respective degrees, then replay the chosen rational inputs. Presenting the coefficient indices as user inputs would be a semantic bug. This extension is designed here but not implemented in the recorded corpus.

Finite-alphabet closure is the recommended first Lean milestone. Polynomial input lifting should follow only after the source bridge, coefficient identity, and concrete-input reconstruction have their own tests.

3 Worker B: target-generated pullback ideals

3.1 The goal and the reason for using ideals

Let $R = \mathbb{Q}[x_1, \dots, x_D]$. Each action is a total polynomial map $F_a : \mathbb{Q}^D \rightarrow \mathbb{Q}^D$. The initial state is a polynomial parameterization

$$S : \mathbb{Q}^k \longrightarrow \mathbb{Q}^D,$$

and the target is $q \in R$. We seek

$$\forall \theta \in \mathbb{Q}^k \ \forall w \in \Sigma^*, \quad q(F_w(S(\theta))) = 0. \quad (5)$$

A constant initial state is represented by constant coordinate polynomials. The prototype uses at least one, possibly unused, parameter in such fixtures.

For a map F , write $F^*p = p \circ F$. This pullback is a ring homomorphism:

$$F^*(p + r) = F^*p + F^*r, \quad F^*(pr) = (F^*p)(F^*r).$$

If a family $g_1, \dots, g_r$ vanishes at a state, so does every polynomial combination $\sum h_i g_i$. Therefore the natural object for preserving *vanishing* is the ideal generated by the observations, not their span over constant scalars.

For linear maps and linear observations, Worker A gives a particularly cheap special case. Polynomial closure allows nonconstant multipliers, which is the key additional expressive power.

3.2 Construct the strengthening instead of guessing it

Start with $J_0 = \langle q \rangle$. Given a finite generating set G_i for J_i , set

$$J_{i+1} = J_i + \langle g \circ F_a : g \in G_i, a \in \Sigma \rangle. \quad (6)$$

The result does not depend on the chosen generating set. Indeed, pulling back $\sum h_j g_j$ gives $\sum (h_j \circ F_a)(g_j \circ F_a)$, so generator pullbacks suffice to control every element of the ideal.

Operationally, compute a Gröbner basis, reduce each generator pullback, and adjoin a nonzero remainder. Recompute the basis and restart the scan. Stop when all remainders are zero. The prototype adds one remainder at a time; a batch implementation can add several, but should measure coefficient growth rather than assuming batching is always faster.

```
G := GroebnerBasis([target])
repeat:
  if some g(initial(parameters)) is not the zero polynomial:
    extract and replay an original-target counterexample
    return Counterexample, or Unknown if extraction is cut off
  for each action F and generator g in G:
    (quotients, remainder) := divide(g composed with F, G)
    if remainder is nonzero:
      if budget exhausted: return Unknown
      G := GroebnerBasis(G union {remainder})
      restart
  if all divisions had zero remainder:
    return initial identities, target membership,
      and all generator-pullback multiplier identities
```

Nothing in this loop simultaneously solves for unknown generators and unknown multipliers. Generators are determined by exact symbolic closure; multipliers are polynomial-division witnesses relative to the final fixed set. This is the specific answer to the bilinearity obstruction identified in Forge’s existing structural section [2].

3.3 The certificate and its soundness

A certificate consists of polynomials $g_1, \dots, g_r$, target multipliers h_j , and a matrix of polynomial multipliers $H_{ij}^{(a)}$ for each action. Its three conditions are

$$q = \sum_{j=1}^r h_j g_j, \quad (7)$$

$$g_i \circ F_a = \sum_{j=1}^r H_{ij}^{(a)} g_j \quad (1 \leq i \leq r, a \in \Sigma), \quad (8)$$

$$g_i \circ S = 0 \quad (1 \leq i \leq r). \quad (9)$$

All are polynomial identities over $\mathbb{Q}$. Their checking needs only addition, multiplication, substitution, and exact comparison of coefficient tables.

Theorem 3.1 (Ideal certificate soundness). *Any certificate satisfying (7)–(9) proves (5).*

Proof. Fix arbitrary parameters θ . Initially each g_i vanishes by (9). If all generators vanish at s , evaluating (8) at s shows that all vanish at $F_a(s)$, since each summand has a factor $g_j(s) = 0$. Induction on a word gives simultaneous vanishing after its execution. Evaluating (7) then gives $q = 0$ at the final state. The argument is independent of the chosen parameters. $\square$

The checker need not know what a Gröbner basis is. It need not verify that the reported generators generate the *smallest* closed ideal. It need not verify a derivation of the generators from the original target. Any collection with the three checked properties is sufficient. This removes Gröbner basis correctness from the eventual Lean trusted boundary.

The original target must still be supplied externally to the checker. A worker must not certify a different target by replacing the problem inside its own bundle. The prototype API takes `problem` and `certificate` as separate arguments; saved bundles package both for reproducibility but do not supply source-level authorization.

3.4 Termination and completeness in the stated fragment

Lemma 3.2 (The generated ideal). *The union of the ascending chain (6) is the ideal generated by all target pullbacks:*

$$J_\infty = \langle q \circ F_w : w \in \Sigma^* \rangle.$$

Proof. The target belongs to the word-generated ideal. That ideal is closed under every pullback, because the pullback of a target word is another target word, and pullback distributes over polynomial combinations. Thus every J_i is contained in it. Conversely, repeatedly applying the closure rule places the pullback for every finite word in some stage. Taking generated ideals and unions gives equality. $\square$

Theorem 3.3 (Unbounded target-invariance decision). *For finitely many total polynomial maps over $\mathbb{Q}$, a polynomial initial parameterization, and a fixed polynomial target, exact pullback-ideal closure terminates. It can decide (5), returning a certificate when true and a concrete rational-parameter, finite-word counterexample when false.*

Proof. The ring $\mathbb{Q}[x_1, \dots, x_D]$ is Noetherian, so its ascending chains of ideals stabilize. Each adjoined nonzero normal-form remainder strictly enlarges the current ideal. Every finite Gröbner-basis computation and polynomial division terminates. Thus the uncapped loop reaches a pullback-closed finite generating set G .

If every $g \circ S$ is the zero polynomial, the final divisions provide (7)–(9), proving the target. Otherwise some $g \circ S$ is nonzero. By Lemma 3.2, g is a finite polynomial combination of target pullbacks. At least one of those pullbacks has nonzero composition with S ; otherwise their combination would also be zero. A nonzero polynomial over $\mathbb{Q}$ cannot vanish on every rational tuple. Hence there is a finite word and a rational parameter tuple at which the original target is nonzero. Enumerating words and finite integer grids finds such a tuple after finitely many steps. Conversely, if (5) holds, each target pullback composed with S vanishes on all $\mathbb{Q}^k$, hence is identically zero, because $\mathbb{Q}$ is infinite. Every element of the generated ideal therefore has zero composition with S . No nonzero initial residual can appear and the algorithm returns a certificate. $\square$

This is a decision theorem for a *fixed equality invariant of an unguarded polynomial system*. It is not a general decision procedure for program verification, reachability of a specified point, inequalities, arbitrary recursive functions, rational maps with poles, or integer division. It does not compute every invariant a user might want. Noetherianity supplies no useful small practical bound on the number or size of intermediate generators. The prototype therefore caps extensions and degrees and may return UNKNOWN on problems the idealized procedure would eventually decide.

3.5 Counterexamples must refer to the original target

A failed initial check of an auxiliary g is not itself the counterexample the user asked for. The useful report gives parameters and an actual action word such that $q(F_w(S(\theta))) \neq 0$. The implementation tracks a conservative pullback-depth bound through ideal extensions and searches target words up to that bound. A Gröbner-basis change does not increase the bound: its generators are combinations of the previous generators. An adjoined pullback increases it by at most one.

For a nonzero residual $r(\theta_1, \dots, \theta_k)$ with coordinate degrees d_i , the grid

$$\{0, \dots, d_1\} \times \dots \times \{0, \dots, d_k\}$$

contains a nonzero evaluation. This follows by applying the univariate root bound successively in each variable. The grid can still be large, so both word and point counts have explicit budgets. Exhausting either yields UNKNOWN, not an unverified counterexample. A final independent checker substitutes the reported integers into the initial map, executes the original polynomial maps, and evaluates the original target using Fraction alone.

3.6 Worked example: a nonlinear relation requiring two invariants

Consider

$$F(x, y, z, w) = (y^2, x, w^2, z), \quad S(a, b) = (a, b, a, -b),$$

and the target $q = x - z$. The initial internal states need not agree: $y = b$ and $w = -b$. Consequently $y = w$ is not a valid strengthening. Merely generalizing the target to equality of the whole pair would make a true theorem false.

Pullback closure discovers

$$g_1 = x - z, \quad g_2 = y^2 - w^2.$$

Their preservation equations are

$$\begin{aligned} g_1 \circ F &= g_2, \\ g_2 \circ F &= x^2 - z^2 = (x + z)g_1. \end{aligned}$$

Both vanish initially. In matrix notation the generator vector transforms by

$$\begin{bmatrix} g_1 \circ F \\ g_2 \circ F \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ x+z & 0 \end{bmatrix} \begin{bmatrix} g_1 \\ g_2 \end{bmatrix}.$$

The multiplier $x+z$ is not constant. A conservation-only worker cannot certify this displayed generator vector, and a constant-coefficient invariant transition does not express its second equation. The ideal worker obtains both the strengthening and its nonlinear transition certificate directly from the target.

The recorded search makes one strict ideal extension and three pullback reductions. Its saved generator order is g_2, g_1 , because that is the selected Gröbner order, rather than the expository order above. The checker does not care about this presentation choice; it checks all indices against the actual order in the certificate.

Changing the third update to $w^2 + 1$ gives a counterexample at the first step. Changing the initial fourth coordinate to $b + 1$ also produces a concrete counterexample. These controls ensure that initial parameterization and update semantics are part of the checked problem, not decorative metadata.

3.7 Further nonlinear examples

For $q = xy - z$, let the actions independently square or cube all three coordinates. Then

$$q(x^2, y^2, z^2) = (xy + z)q(x, y, z),$$

and

$$q(x^3, y^3, z^3) = (x^2y^2 + xyz + z^2)q(x, y, z).$$

Starting from (a, b, ab) , the relation holds under every interleaving of these actions. The degrees of explicitly unfolded trajectories grow exponentially with word length, but the certificate remains a small pair of polynomial identities. Four scaled versions are included in the corpus.

Cassini's relation admits a similarly compact state representation. With

$$F(x, y, t) = (y, x + y, -t), \quad S = (0, 1, -1), \quad q = x^2 + xy - y^2 - t,$$

we have $q \circ F = -q$ and $q(S) = 0$. The sign coordinate makes the changing right-hand side part of the state, avoiding any need for a separate transcendental or exponential representation. A mutant that fails to negate t is rejected by a concrete trace.

The chain fixtures require several automatically discovered invariants. Two synchronized length- d shift registers rotate their coordinates and square the returning coordinate. Only equality of their first coordinates is supplied as the goal. Closure discovers equality of all coordinates, requiring $d - 1$ strict ideal extensions in the recorded examples. This tests recursive strengthening rather than merely recognizing a planted one-generator factor.

3.8 Where the theorem stops

Guards. Replacing guarded updates by always-enabled maps is an overapproximation. A successful invariant proof remains sound for the guarded system. A counterexample may traverse an impossible branch and therefore must be replayed against the original guards before being reported. The completeness theorem does not apply to that overapproximation as a decision procedure for the original guarded system.

Initial equations instead of parameterizations. If initial states are given by equations $e_i = 0$, ideal-membership certificates for each generator are sufficient. They need not be complete

for vanishing on real solutions. Real radical issues must not be concealed behind an ordinary Gröbner-basis call. Polynomial parameterizations avoid that particular semantic gap in the theorem proved here.

Division and truncated operations. A rational update needs denominator nonvanishing obligations or a totalized semantics with explicit cases. Lean natural subtraction and integer division are not ordinary polynomial ring operations. Reifying them as such is unsound. The first implementation should reject them rather than accumulate ad hoc exceptions.

Termination. An invariant certificate proves a property of every finite execution. It does not show that a while loop terminates or that a recursive definition is accepted by Lean. Any termination argument remains a separate obligation.

Cost. The prototype’s budgets are checked between symbolic operations. They cannot interrupt an individual expensive Gröbner computation. A production worker needs process-level time and memory limits as well as algebraic limits on degree, term count, and coefficient size.

4 Worker C: pole-free symbolic summation certificates

4.1 Why a rational certificate is not the whole proof

A Gosper or creative-telescoping calculation can produce a rational function that makes a summand telescope away from its poles. The exceptional points may be exactly the endpoints of the finite sum. Cancelling symbolic denominators and then substituting those endpoints is not a proof of the original sum. SymPy provides a useful Gosper implementation for exploration, while more general telescoping algorithms and Ore-algebra libraries supply broader proposal mechanisms [14, 12, 15]. Forge should accept the resulting identity only with its domain and boundary obligations.

The implemented worker deliberately chooses a smaller domain in which the certificate can be made polynomial from the outset. It handles

$$S_{m,W}(n) = \sum_{k=0}^n W(k) \binom{n}{k}^m, \quad m \geq 1, \quad W \in \mathbb{Q}[k]. \quad (10)$$

The checker accepts powers $1 \leq m \leq 6$ within its resource limits. The recorded successful searches use $m = 1, 2$ and polynomial moments $W(k) = k^j$. The family does not presently include $W(n, k)$: that extension requires shifting its first argument in the $n + 1$ summand, and cannot be simulated by keeping the same W on both sides.

4.2 A boundary-safe binomial basis

For nonnegative integers n, k , interpret the following quantities in $\mathbb{Q}$:

$$B(n, k) = \binom{n}{k}, \quad A(n, k) = \begin{cases} 0, & k = 0, \\ \binom{n}{k-1}, & k > 0. \end{cases}$$

As usual $\binom{n}{k} = 0$ for $k > n$. Two identities hold on the entire nonnegative index domain:

$$\binom{n+1}{k} = A(n, k) + B(n, k), \quad (11)$$

$$(n+1-k)A(n, k) - kB(n, k) = 0. \quad (12)$$

Here $n+1-k$ is subtraction after embedding in $\mathbb{Q}$, not truncated subtraction in $\mathbb{N}$. For $k > n+1$, both binomial factors vanish. At $k = 0$, the separate zero definition of A is essential. Defining

$A(n, 0)$ using `Nat.choose n (0 - 1)` would give $\binom{n}{0} = 1$, because natural subtraction truncates. That is precisely the wrong endpoint.

The pinned Mathlib source provides `Nat.choose`, the successor/Pascal identity, and vanishing above the top index [18]. A Lean bridge should prove (11) and (12) once, splitting $k = 0$, $1 \leq k \leq n + 1$, and $k > n + 1$ as needed. These primitive theorems can then be reused for every certificate in the family.

4.3 The exact certificate language

Search for $p_0, p_1 \in \mathbb{Q}[n]$, a polynomial $r \in \mathbb{Q}[n, k]$, and a polynomial $h \in \mathbb{Q}[A, B, n, k]$ such that

$$p_1(n)W(k)(A+B)^m + p_0(n)W(k)B^m - r(n, k+1)B^m + r(n, k)A^m = h(A, B, n, k)((n+1-k)A - kB), \quad (13)$$

with $p_1 \neq 0$. Every symbol here is a polynomial indeterminate. In particular, the coefficient field is $\mathbb{Q}$, not $\mathbb{Q}(n, k)$. Division by a polynomial in n or k is not allowed to hide inside the coefficient domain.

The certificate stores the four polynomials p_0, p_1, r, h . The problem supplies m and W . The checker independently expands both sides of (13) by sparse rational arithmetic. It does not need to run Gosper's algorithm, test a rational-function identity, or trust a simplifier's handling of singular values.

Theorem 4.1 (Pole-free sum recurrence). *If (13) holds for $m \geq 1$, then*

$$p_1(n)S_{m,W}(n+1) + p_0(n)S_{m,W}(n) = 0 \quad (n \in \mathbb{N}). \quad (14)$$

Proof. Substitute $A = A(n, k)$ and $B = B(n, k)$ into the polynomial certificate. The right-hand side vanishes by (12). Define

$$T(n, k) = r(n, k)A(n, k)^m.$$

Since $A(n, k+1) = B(n, k)$, the remaining identity is

$$p_1(n)W(k)\binom{n+1}{k}^m + p_0(n)W(k)\binom{n}{k}^m = T(n, k+1) - T(n, k).$$

Sum this equality for $k = 0, \dots, n+1$. The first term is exactly $p_1(n)S_{m,W}(n+1)$. The extra $k = n+1$ term from the second row is zero, so the second term is $p_0(n)S_{m,W}(n)$. The right side is $T(n, n+2) - T(n, 0)$. Both endpoints vanish: $A(n, 0) = 0$ and $A(n, n+2) = \binom{n}{n+1} = 0$, and $m \geq 1$. This proves the recurrence without any denominator cancellation. $\square$

The condition $m \geq 1$ is mathematical, not just a convenient validation rule. For $m = 0$ the endpoint factors would be $0^0 = 1$ in the usual algebraic power convention, so the argument would not establish zero boundary terms. The checker rejects that input rather than silently reusing the theorem.

4.4 Synthesis by a finite nullspace computation

Choose degree bounds d_p, d_r and the templates

$$p_i(n) = \sum_{j=0}^{d_p} c_{ij}n^j, \quad r(n, k) = \sum_{a+b \leq d_r} d_{ab}n^a k^b.$$

There are

$$N = 2(d_p + 1) + \binom{d_r + 2}{2}$$

unknown coefficients. The residual on the left of (13) is linear in these unknowns.

Reduce each coefficient column modulo the principal ideal generated by $\rho = (n + 1 - k)A - kB$ in $\mathbb{Q}[A, B, n, k]$. Collect the coefficients of all remaining monomials into an exact rational matrix M . A vector in $\ker M$ describes a residual that lies in $\langle \rho \rangle$. Retain a vector only if its p_1 component is nonzero. Multiply by a common denominator, remove the integer content, and choose a positive leading coefficient of p_1 for a deterministic display. Finally, divide the completed residual by ρ to obtain h and replay (13) independently.

Proposition 4.2 (Fixed-template coverage). *Exact nullspace computation finds a nontrivial order-one certificate of this form whenever one exists within the chosen p_i and r templates.*

Proof. Normal-form reduction by a fixed Gröbner basis is $\mathbb{Q}$ -linear. Thus the matrix kernel is exactly the coefficient vectors whose residual has zero normal form, which is exactly membership in the principal ideal. The projection of a kernel basis onto the p_1 coefficients is nonzero precisely when that projection is nonzero for some kernel vector. Hence inspecting a full kernel basis suffices to find a permitted nonzero p_1 component. $\square$

The synthesis loop tries degree pairs in increasing total template cost. It is not a general creative-telescoping decision procedure: it fixes recurrence order one, a particular pole-free certificate form, and finite degree bounds. A missing certificate means UNKNOWN for the sum identity, even if the finite linear template search was fully exhausted. The successful larger-template runs and the unsuccessful smaller-template runs are both retained in the experimental corpus.

4.5 Worked example: the squared-binomial identity

For $m = 2$, $W = 1$, the search produces

$$p_1 = n + 1, \quad p_0 = -2(2n + 1), \quad r = 2k - 3n - 3, \quad h = 2(B - A).$$

The complete local certificate is

$$\begin{aligned} & (n + 1)(A + B)^2 - 2(2n + 1)B^2 \\ & - (2(k + 1) - 3n - 3)B^2 + (2k - 3n - 3)A^2 \\ & = 2(B - A)((n + 1 - k)A - kB). \end{aligned}$$

It can be verified by an ordinary ring normalization. Theorem 4.1 then gives

$$(n + 1)S(n + 1) = 2(2n + 1)S(n), \quad S(0) = 1.$$

The sequence $H(n) = \binom{2n}{n}$ has the same initial value and recurrence: for example its factorial ratio gives

$$\frac{H(n + 1)}{H(n)} = \frac{(2n + 2)(2n + 1)}{(n + 1)^2} = \frac{2(2n + 1)}{n + 1},$$

where the positive integer denominators are nonzero. Since $n + 1 \neq 0$ on $\mathbb{N}$, induction establishes

$$\boxed{\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}}.$$

This last comparison is a mathematical derivation in the article. The executed Python certificate certifies the recurrence, not a Lean theorem about the closed form.

A rational Gosper representation of the same telescoping term is

$$\frac{k^2(2k - 3n - 3)}{(n + 1 - k)^2} \binom{n}{k}^2.$$

Its denominator vanishes at $k = n + 1$. Rewriting the *entire term* as $(2k - 3n - 3)A(n, k)^2$ gives a value on that boundary and permits a direct pointwise proof there. This is not the assertion that two partial rational expressions agree at a pole. It is a new total expression accompanied by an identity valid at every required index.

For $m = 1$, $W = 1$, the search finds $p_1 = 1$, $p_0 = -2$, $r = -1$ and the primitive relation multiplier $h = 0$. The corresponding recurrence is $S(n + 1) = 2S(n)$; with $S(0) = 1$ it gives the usual 2^n sum.

4.6 Initial values and singular leading coefficients

A recurrence is not a uniqueness theorem without a nonvanishing condition or additional seeds. The first-moment case produces

$$nS(n + 1) - 2(n + 1)S(n) = 0.$$

At $n = 0$, this equation forces $S(0) = 0$ but says nothing about $S(1)$. The true sequence $S(n) = \sum_k k \binom{n}{k}$ and the wrong sequence $2S(n)$ satisfy the same recurrence and the same $n = 0$ seed. The regression suite explicitly checks this counterexample to a base-only acceptance rule.

Proposition 4.3 (Order-one recurrence comparison). *Suppose u, v satisfy the same recurrence $a(n)z(n + 1) + b(n)z(n) = 0$ on $\mathbb{N}$. Let $E = \{e \in \mathbb{N} : a(e) = 0\}$. If $u(0) = v(0)$ and $u(e + 1) = v(e + 1)$ for every $e \in E$, then $u = v$ on $\mathbb{N}$.*

Proof. Induct on n . To obtain equality at $n + 1$, use the corresponding extra seed when $a(n) = 0$. Otherwise subtract the two recurrence equations, use equality at n , and cancel the nonzero scalar $a(n)$. $\square$

For nonzero rational-polynomial a , E is finite. A production comparator can clear constant denominators, remove the power of n , and enumerate possible integer roots using divisibility of the constant coefficient. Each proposed root is checked by evaluation, and the divisibility argument accounts for all remaining roots. A simpler but potentially expensive checker can use a Cauchy root bound and evaluate every integer below it. Either route requires a proof of coverage; a CAS list of roots is not itself that proof.

The prototype records nonnegative roots of p_1 as *search diagnostics*. The independent recurrence checker does not use those diagnostics to accept a closed form. The Lean recurrence-comparison layer and certified seed-coverage check are proposed work, not implemented artifacts.

4.7 The seven synthesized recurrences

Table 2 reports the actual recurrences, with $S_{m,j}(n) = \sum_{k=0}^n k^j \binom{n}{k}^m$ and $k^0 = 1$. They all have the form $p_1(n)S_{m,j}(n + 1) + p_0(n)S_{m,j}(n) = 0$. Common factors are not silently cancelled, because cancellation could alter the exceptional-index obligations.

Table 2: Synthesized recurrence coefficients and total degree of $r(n, k)$, generated from the saved certificates.

m	j	$p_1(n)$	$p_0(n)$	$\deg r$
1	0	1	-2	0
1	1	n	$-2(n+1)$	2
1	2	n	$-2(n+2)$	3
1	3	$n^2(n+3)$	$-2(n+1)^2(n+4)$	6
2	0	$n+1$	$-2(2n+1)$	1
2	1	n^2	$-2n(2n+1)$	3
2	2	n^3	$-2(n+1)^2(2n-1)$	5

The two highest-moment searches fail under $d_r \leq 4$ and succeed when the explicitly stated bound is enlarged. An order-one attempt for $m = 3, W = 1$ remains UNKNOWN at the recorded small bounds. No claim is made that the sequence lacks a higher-order recurrence or that the search has exhausted creative telescoping in general.

4.8 A route to general creative telescoping

The next interface should accept a telescoper $L = \sum_{i=0}^{\ell} p_i(n) E_n^i$, a finite set of pointwise summand relations, and a certificate for

$$\sum_{i=0}^{\ell} p_i(n) f(n+i, k) = G(n, k+1) - G(n, k).$$

The acceptance pipeline must also prove a common summation range, any added or removed tail terms, endpoint values of G , and enough initial data to compare the resulting recurrence with the target sequence. Nonzero denominators become explicit subgoals; singular indices become separate branches. When endpoints are not zero, their exact difference belongs on the right-hand side of the recurrence rather than being discarded.

A Sage `ore_algebra` process is a concrete optional proposal backend: its documented functionality includes creative telescoping, desingularization, and operations on D-finite descriptions [15]. The compact certificate methods of Brochet and Salvy provide an additional algorithmic reference [12]. Neither is a Lean oracle. Their output still has to be translated into the pointwise relations and boundary certificate accepted by Forge. This interface is designed here, whereas the delivered code implements only the polynomial binomial template.

5 Integrating the workers into Forge and Leant

5.1 The missing component is a semantic compiler

The mathematical certificate checkers are intentionally small. The more subtle Lean work is showing that the reified problem denotes the original goal. A correct polynomial identity about an incorrectly translated expression proves nothing useful. Forge’s own contract already separates certificate soundness from this correspondence [5]; the new contribution is to make the required correspondence concrete for these workers.

For a source state type S , input type A , and original update step $: S \rightarrow A \rightarrow S$, a positive bridge

should contain:

$$\begin{aligned}
 e &: S \rightarrow \mathbb{Q}^D, \\
 \iota &: A \rightarrow \Sigma, \\
 e(\text{step}(s, a)) &= F_{\iota(a)}(e(s)), \\
 e(\text{initial}(\theta)) &= S_0(\theta), \\
 q(e(s)) = 0 &\implies \text{Goal}(s).
 \end{aligned}$$

Each displayed equality or implication is a Lean theorem, not a callback assertion. These facts, the certificate theorem, and induction on the original input list produce the source goal. Injectivity of e is not required when the observation implication already gives exactly the desired proposition.

A negative bridge needs more. A rational state or action produced by a solver might have no source inhabitant. Thus a counterexample becomes a user-visible refutation only after constructing the original typed parameter values and inputs and checking the original proposition at that concrete execution. A positive simulation into an overapproximation is insufficient to justify lifting a negative result back.

5.2 An admission algorithm for numeric folds

The first reifier should target explicit folds over lists and a small first-order state grammar. It should not begin by trying to translate arbitrary Lean metaprograms or dependent recursive definitions. A practical admission pass is:

1. Normalize only the approved definitional equations of the fold and its step function. Preserve lets until their dependencies and sharing are recorded.
2. Collect scalar state fields and parameters. Admit rational ring operations and natural exponents. Translate each expression by an interpretation theorem, so addition, multiplication, constants, and projection all come with source correspondence.
3. Classify state dependence. Degree at most one yields Worker A after homogenization; general admitted polynomial dependence yields Worker B.
4. Enumerate an actual finite input type, or use a supplied exhaustive classification theorem. Merely seeing a few constructors in samples does not establish exhaustive action coverage.
5. Build the target observation and initial parameterization. Preserve casts, state-field order, and every parameter dependency explicitly.
6. Submit the exact reified problem. Replay the returned algebraic identities and compose the semantic bridge with the source fold-induction theorem.

Every rejected construct should be named: for example, “state-dependent integer division”, “unknown numeric dictionary”, or “input alphabet not proved exhaustive”. Falling back to Forge’s existing engines is legitimate; pretending the unsupported expression is polynomial is not.

A source fold may have a nonnumeric payload alongside a numeric accumulator. It can be erased only when a theorem establishes that the payload does not affect the observation or the numeric update. If it controls a branch or carries a typeclass method, it cannot be ignored. This is where a seemingly convenient “just extract the arithmetic” implementation often becomes unsound.

5.3 Universal behavior as a new Leant verification branch

Leant already checks an exact candidate against its type and then tries to prove or refute a supplied behavioral proposition using `decide` and bounded `simp`. Its documentation carefully distinguishes a conjunction of examples from a universal assertion, and reports inconclusive checks rather than

inventing a counterexample [7]. The proposed addition is a restricted universal-behavior branch, not a replacement for that discipline.

For a candidate implementing a numeric fold and a reference specification, the adapter should compile both to the admitted state model, form the product observation, and ask Worker A or B to prove it zero for all input words. On success, it reconstructs a theorem about the *same exact candidate*. On failure, a concrete word can become a new rejecting behavioral example after source-level replay. On UNKNOWN, the candidate remains inconclusive for the universal assertion.

This can improve synthesis in two different ways. A universal certificate can accept a candidate that finite executable tests cannot establish correct. Conversely, a distinguishing word can reject many other candidates cheaply, provided each candidate is separately evaluated at that word. The latter is ordinary counterexample-guided filtering; it does not transfer one candidate’s proof or rejection receipt to another.

The adapter should retain the already verified source identity, lexical universe choices, and dictionaries. Djex and Leant invest substantial effort in preserving precisely those associations [8, 6]. This article does not propose to rebuild their existing ownership or rank-N search mechanisms. It adds a mathematical behavior verifier behind that boundary.

5.4 A warning about Church encodings and parametricity

A proof about folds of genuine finite lists does not automatically prove a statement about every inhabitant of a Church-encoded type. In Lean, one cannot silently import a Haskell-style free theorem, erase a universe distinction, or assume that every polymorphic value is the image of an ordinary list encoding. Those claims need their own source-level theorems and hypotheses.

The first integration should therefore accept either an explicit `List` fold or a Church value accompanied by a proved representation theorem. A certificate for all encoded finite lists should be displayed at that scope, not as an answer to a more general original quantified signature. In particular, this proposal does not close the neighboring projects’ outstanding integer indexing or simultaneous-universe composition queries merely by solving easier numeric fold examples [6, 8].

5.5 Certificate-guided, but not heuristic, semantic pruning

Once universal equivalence is available for a restricted grammar, it can safely remove redundant candidate implementations *under an explicitly proved observational congruence*. Suppose two complete candidates have the same numeric output on every admissible input. They are interchangeable for a final specification that observes only that output. They are not automatically interchangeable as arbitrary higher-order subterms, because an enclosing context might inspect their representation, intermediate state, dictionaries, or a second output.

A safe first optimization deduplicates only *complete candidates at the root behavioral specification*, keeping one representative and its own source receipt. A more ambitious grammar-level quotient needs a congruence theorem for every grammar constructor. It must also preserve a representative for each required source type and dictionary instantiation. The proposed workers supply equivalence evidence; they do not make an arbitrary lossy hashing scheme safe. This optimization is not part of the executed prototype.

5.6 The first Lean implementation should emit small proofs

There are two sensible replay paths. The initial path emits ordinary Lean proofs of the finitely many certificate identities, using existing arithmetic normalization and explicit substitution lemmas. The later path uses a reflected sparse-polynomial checker with a proved evaluation

theorem. Reflection is a performance and proof-size improvement, not a prerequisite to validate the first feature.

For Worker A, emit proofs of $q = tG$, $GA_a = C_aG$, and $Gs_0 = 0$. Apply a single generic invariant theorem. For Worker B, prove the polynomial identities in (7)–(9), evaluate them at a state, and apply simultaneous invariant induction. The mathematical derivation of a Gröbner basis need not appear in the Lean proof. For Worker C, prove the local polynomial identity, apply the two binomial bridge lemmas, telescope a finite sum, and only then address recurrence comparison and exceptional seeds.

The supplied `lean/ForgeExtension/Core.lean` is the semantic list-execution induction glue. `Worked.lean` contains a hand-written replay target for the mutually supporting nonlinear invariants and the local squared-binomial identity. They contain no `sorry`, declared axiom, or native-evaluation shortcut, but they were *not compiled*. Their purpose is to pin down the intended theorem shapes, not to pretend that the full reifier, reflection theorem, or tactic exists.

5.7 Concrete libraries and module responsibilities

Component	Concrete choice	Required boundary
Linear search	SymPy <code>Matrix.rank</code> , <code>gauss_jordan_solve</code> ; later an incremental echelon implementation	Exact rational coefficients only; no numeric-rank acceptance.
Ideal search	SymPy <code>groebner</code> over <code>QQ</code> , graded reverse lexicographic order, <code>GroebnerBasis.reduce</code>	Export multiplier identities; the checker does not trust basis claims or normal-form status.
Binomial synthesis	SymPy polynomial reduction and <code>Matrix.nullspace</code>	Polynomial coefficient domain $\mathbb{Q}[A, B, n, k]$; retain endpoint-safe form.
Independent replay	Python standard library <code>fractions</code> , <code>json</code> , explicit sparse operations	No import of search code or symbolic library.
Lean polynomial semantics	<code>Mathlib.Algebra.MvPolynomial.Eval</code>	Evaluation, substitution, and ring homomorphism lemmas connect identities to source expressions.
Lean matrix semantics	<code>Mathlib.Data.Matrix.Mul</code>	Matrix, vector, and dot-product conventions must agree with the serialized row/column layout.
Lean binomial semantics	<code>Mathlib.Data.Nat.Choose.Basic</code>	Prove the zero-extended predecessor convention and rational casts explicitly.
General telescoping, optional	Sage <code>ore_algebra</code> as an external proposer	Only source-grounded pointwise identities and checked boundaries have proof authority.

The SymPy API choices used by the prototypes are documented, and its polynomial reduction returns quotient witnesses as well as a remainder [13]. The named Mathlib source files were inspected at Forge’s pinned revision; `MvPolynomial` provides evaluation maps, and the matrix file explicitly specifies its row/vector conventions [16, 17]. These are concrete integration targets, not claims that a compatible binary build was available in the preparation environment.

The source package pins the same reference Lean and Mathlib revisions as Forge for the uncompiled specimen project. A first real integration run must compile against that pin, inspect all resulting axioms under Forge’s existing policy, and record the exact environment. Reusing Forge’s policy is preferable to inventing another almost-identical trust profile in this extension.

6 Implementation and experimental evidence

6.1 What was executed

The archive contains two intentionally different implementations of the mathematical operations. `prototype/search.py` uses SymPy 1.14.0 for exact matrix solving, polynomial bases, division, and nullspaces. `prototype/checkers.py` uses only the Python standard library, representing polynomials as sparse exponent maps with `Fraction` coefficients. The checker neither imports the search module nor calls a symbolic simplifier. It can replay saved bundles with Python’s site packages disabled.

The environment was Python 3.13.5 on x86-64 Linux. The deterministic corpus seed is 2026091507. Search time is measured once per case with `perf_counter_ns`; checker time is the median of five warm invocations. Process startup, imports, JSON parsing, and any Lean work are excluded from those timings. This is a small deterministic research corpus, not a production performance study.

Worker	Cases	Cert.	Cex.	Unknown	Search (ms)	Check (ms)
Observable closure	36	22	14	0	3.098	0.225
Pullback ideals	13	9	3	1	6.306	0.179
Binomial sums	10	7	0	3	47.632	0.313

Table 4: Recorded outcomes. Timing columns are medians over the successful instances in each family; the checker entry for each instance is itself a median of five warm runs. Cex. means independently replayed counterexample.

Across 59 cases, 38 yield certificates, 17 yield concrete counterexamples, and four retain UNKNOWN. The unknown cases are expected budget or template controls, not failed cases reclassified as successes. The exact per-instance results, options, and timings are in `results/summary.json`, `results/problems.json`, and `results/measurements.csv`.

6.2 Corpus construction and what each part tests

The 36 linear cases comprise 18 exact invariant systems in dimensions two through seven, four product-equivalence systems related by invertible changes of coordinates, seven delayed-error shift systems, six guaranteed nonzero random controls, and a dedicated noncommuting two-action orientation test. The positive invariant fixtures deliberately contain unobserved coordinates; this tests demand-directed closure rather than forcing the basis to equal the entire state space.

The 13 polynomial cases comprise the mutual-squares example, its budget control, two mutants, four scaled product relations under square/cube actions, a Cassini relation and its mutant, and three strengthening chains. The target-only initial ideals are not supplied with the final auxiliary generators. In the largest chain, five generators are discovered after four strict extensions.

The ten summation cases comprise seven successful binomial-moment recurrences, two smaller-template versions that remain unknown, and a bounded order-one attempt for the cubic-binomial sum. The numerical cross-checks evaluate each accepted recurrence for $n = 0, \dots, 30$. Those

evaluations are independent consistency tests; the universal argument is the polynomial certificate plus Theorem 4.1, not those 31 values.

No instance was imported from a benchmark and then represented as unseen. These cases are intentionally constructed to distinguish exact scope, boundary behavior, and implementation failure modes. They do not estimate coverage on arbitrary Lean developments. Likewise, the linear and polynomial case counts are not compared with the many different historical runs already stored in Forge’s repository.

6.3 Selected measurements and certificate sizes

The largest product-equivalence example has total state dimension ten and an observable basis of size five. The search processes eleven queued rows. Its recorded search time is 18.576 ms and median checker time 1.785 ms. The mutual-squares ideal example takes 10.862 ms to search and 0.214 ms to replay; its complete compact problem-and-certificate JSON bundle is 562 bytes.

The length-five polynomial strengthening chain uses five generators and 15 pullback reductions. Its search takes 55.934 ms and replay 0.810 ms. The squared-binomial zeroth moment is found after five template trials with seven unknown columns; its recorded search is 19.534 ms and replay 0.261 ms. The binomial third moment requires total certificate degree six in the successful run and takes 220.754 ms to search.

All 38 successful compact JSON bundles total 27,814 bytes before ZIP compression. The raw column named `certificate_bytes` counts the entire serialized bundle, including its problem and case name; it is not the size of a Lean proof term or the certificate payload alone. Actual Lean reconstruction and kernel-check costs remain unmeasured.

These small measurements support one modest conclusion: for this corpus, finding an explanation is more expensive than replaying its finite identities, and the saved explanations are compact. They do not support a general speedup claim, an asymptotic scaling claim about SymPy’s implementation, or a comparison with `grind`.

6.4 Independent replay, mutations, and schema controls

The command

```
python -S prototype/checkers.py results/certificates results/counterexamples
```

replays 55 bundles: 38 invariant or recurrence certificates and 17 concrete counterexamples. The retained output reports zero failures and `site_packages_enabled: false`. The four unknown outcomes have no proof or counterexample certificate and are not included in this replay count.

For every successful certificate, the mutation harness adds one to each stored matrix entry or nonzero polynomial coefficient, one position at a time. All 818 resulting mutations are rejected. Some mutations create a noncanonical explicit zero monomial, while others preserve the encoding and break an algebraic identity. This is a finite adversarial corpus, not a proof that every possible forgery is rejected.

The 117 pytest items additionally cover target binding, altered initial states, missing action coverage, duplicate JSON keys, floating-point tokens, bool-as-integer confusion, unreduced rationals, duplicate or unordered monomials, negative exponents, excessive declared degree, empty linear bases, word direction, a nonlinear budget cutoff, zero binomial endpoints, and the singular-recurrence nonuniqueness example. One item performs 60 independently seeded sparse-arithmetic differential trials against SymPy. These are different units: 117 test items are not 117 distinct mathematical theorems, and 818 mutations are not 818 new source goals.

6.5 Failures retained rather than hidden

Development exposed two substantive mistakes in the initial test expectations. First, a randomly generated matrix system was labeled negative without ensuring that its target ever became nonzero. The solver correctly certified it. The fixture generator was repaired to force an observed nonzero update, and a separate length-two case was added for word ordering. This was a fixture error, not a failed mathematical certificate.

Second, the first moment-three summation expectation assumed that a degree-four r template would suffice. It did not. The smaller-template unknown remains in the final corpus; the successful run explicitly permits degree six. The same paired control was added for the squared-binomial second moment. Reporting only the later successful run would obscure the actual search limitation.

Two test-development failures are also logged: a collection-time bracket error, and an assertion that incorrectly required both recurrence coefficients to vanish at the singular index. Only the leading coefficient needs to vanish to lose the next-value constraint; the other coefficient can multiply a zero current value. The corrected regression still checks two different sequences with the same recurrence and first seed.

The retained diagnostic files explain these changes. They are not counted as accepted runs, and they do not contain full historical source snapshots. `docs/validation.md` states that distinction explicitly.

6.6 Limitations of the executed artifact

The Python checker has input limits: at most 16 scalar dimensions, bounded integer encodings, at most 20,000 monomials per admitted polynomial, degree at most 128, and a file-size ceiling. It checks intermediate polynomial growth as well. These are pragmatic rejection conditions, not a formal resource theorem. External process isolation remains necessary for adversarial use, particularly because exact arithmetic and symbolic search can consume significant memory before the next boundary check.

The supplied Lean code was not compiled, the source reifier was not built, and none of the source-to-IR bridges was machine-verified. No claim is made that 117 Python tests remove the possibility of a checker bug. What is available now is a reproducible mathematical implementation and a small, sharply specified set of theorems that the Lean implementation should prove.

7 A concrete implementation sequence

7.1 Milestones with acceptance gates

The first production objective should be *one source-level theorem closed end to end*, not a dashboard that can dispatch to every proposed worker. The following sequence keeps the new work separable and measurable.

Stage	Deliverable	Acceptance gate
A1	Linear semantic theorem and certificate replay	At the pinned Lean revision, prove the matrix certificate theorem, compile positive and negative replay fixtures, and record the permitted axioms.
A2	Explicit rational fold reifier	Reconstruct the bit-accumulator example from its original fold definitions; reject integer division and nonexhaustive inputs.
A3	Observable search integration	Load a certificate from an untrusted process, close the exact source goal, and replay a distinguishing word on the original source function.
B1	Polynomial certificate theorem	Check the mutual-squares and product examples using supplied multipliers; no Gröbner computation inside the trusted proof boundary.
B2	Target-generated ideal search	Discover the auxiliary generator without a hint; preserve unknown on the extension-budget control; reject both source mutants.
C1	Binomial primitive bridge and finite telescoping	Prove predecessor-zero semantics, cross multiplication, support, and endpoints; close the squared-binomial recurrence in Lean.
C2	Recurrence comparison	Close the central-binomial closed form; reject the false base-only proof for the first moment until its additional seed is provided.
L1	Leant behavior adapter	Check an exact candidate universally at its original permitted specification; keep inconclusive outcomes and all source identities intact.

These are implementation gates, not calendar promises. The source bridge and Lean replay are the critical path. Building a second symbolic algebra engine is not. SymPy can remain the first untrusted proposer while the Lean layer matures.

7.2 Suggested module decomposition

The names below are proposed additions, not claims about modules already present in Forge:

```

Forge/Behavior/StepIR.lean      -- exact scalar-state model
Forge/Behavior/FoldBridge.lean  -- source simulation theorems
Forge/Observable/Certificate.lean -- G, t, and action matrices
Forge/Observable/Replay.lean    -- finite matrix identity proofs
Forge/Invariant/IdealCert.lean  -- generator/multiplier identities
Forge/Invariant/Replay.lean     -- simultaneous invariant induction
Forge/Summation/BinomialBridge.lean
Forge/Summation/Telescope.lean
Forge/Summation/RecurrenceCompare.lean
Forge/Behavior/LeantAdapter.lean -- exact-candidate behavior hook

```

The search side should serialize canonical problem data rather than executable Lean source. Initially a Lean-side decoder can reconstruct explicit literals and invoke existing proof-producing

normalizers. Once profile data shows proof construction dominating, replace repeated normalizations with a reflected checker while leaving the problem schema and semantic theorems unchanged.

Do not add all discovered auxiliary invariants as mutually assumed lemmas. Instead, prove the conjunction inductively with one simultaneous invariant theorem and only then publish its consequences. A cyclic preservation matrix is sound because the induction hypothesis makes the entire previous-state vector available. It is not permission to accept a circular proof graph without induction.

7.3 A cheap exact adapter for polynomial targets of affine systems

There is a useful middle ground between Workers A and B. If every update is affine and the target has degree at most d , collect all monomials in D variables of total degree at most d . Their number is

$$N = \binom{D+d}{d}.$$

Substitution of an affine map preserves this degree bound. Hence the monomial vector $v_d(s)$ obeys a finite linear update

$$v_d(F_a(s)) = L_a v_d(s)$$

for exactly computable rational matrices L_a . Express the target as a row in this basis and use Worker A in dimension N .

The adapter requires a checked monomial-evaluation identity for each matrix L_a , then reuses the existing observable certificate theorem. It avoids Gröbner search and is complete for this fixed-degree polynomial target under affine updates. It can nevertheless be impractical when N is large; the classifier should estimate N before materializing the lifted matrices. This is a concrete proposed optimization, not an executed fourth worker in the reported counts.

7.4 Multiple control locations without inventing cyclic proofs

For an unguarded finite control-flow graph, assign an ideal J_v to each location v . For an edge $e : u \rightarrow v$ with polynomial update F_e , close the family under

$$g \in J_v \implies g \circ F_e \in J_u.$$

Targets seed the appropriate locations. The finite product of ascending ideal chains stabilizes because each component ring is Noetherian. A certificate for each edge has the form

$$g_{v,i} \circ F_e = \sum_j H_{e,ij} g_{u,j}.$$

Initial generator identities at each entry location and ordinary induction on a finite execution path prove the location-specific observations. Counterexamples must be reconstructed as actual graph paths, not arbitrary words through the edge set.

This is a natural extension of the single-location worker. It adds a finite control index without changing the algebraic leaf checker. With guards, the same warning as in Section 3.8 applies: treating every edge as enabled is sound for positive proofs but can generate infeasible negative traces. The graph extension is not included in the current prototype.

7.5 Selection and performance without another scheduler design

Use Forge’s existing demand and budget mechanism. The new classifier supplies only the domain facts needed to select a worker: state dimension, action count, state degree, target degree, initial-parameter degree, summand template, and presence of unsupported operations. A sensible initial policy is:

1. Try direct existing simplification and known lemmas under the current Forge policy; avoid paying for algebra when a theorem application suffices.
2. Prefer observable closure for linear targets and updates, or an affordable fixed-degree affine lift.
3. Use ideal closure for admitted nonlinear polynomial systems, with explicit extension, degree, term, coefficient, wall-time, and memory limits.
4. Route recognized binomial sums to the pole-free template, increasing bounds within the caller’s existing budget rather than resetting it per trial.

The polynomial and summation searches should cache substitutions and reduced columns under exact problem and basis identities. An ideal-basis change invalidates normal forms from the old basis unless a checked conversion is performed. Summation matrix columns can be extended incrementally as the template grows, but a new coefficient domain or different primitive relation requires a new cache identity.

For linear closure, implement incremental rational echelon reduction before adding complicated portfolio machinery. For ideals, try returning a smaller valid generating family after success, but do not require minimality for acceptance. For large multiplier expressions, share subexpressions in proof construction and measure kernel time separately from search time. These are specific optimization opportunities whose effect can be assessed without changing the mathematical claim proved by a certificate.

7.6 What a convincing comparison should look like

A future Lean evaluation should use identical source theorems, identical library availability, and a fixed versioned environment. It should compare at least: existing Forge behavior; Forge plus the new worker but with supplied invariants; and Forge plus automatic closure. That separation distinguishes the benefit of new certificate replay from the benefit of invariant discovery.

The negative suite should include nonexhaustive alphabets, reversed action words, altered initial maps, unavailable source witnesses, incorrect casts, truncated subtraction at a binomial endpoint, a rational pole at the support boundary, and missing recurrence seeds. Timeouts must stay unknown. Report search, decoding, proof construction, and kernel checking separately. None of these production measurements is replaced by the Python timings in this article.

8 Conclusion

The central recommendation is to make Forge stronger by giving it new finite mathematical explanations, not by adding more unstructured search. Observable closure turns infinitely many linear executions into a small invariant row space. Pullback-ideal closure derives the polynomial strengthening needed by a goal and certifies it with fixed multiplier identities. Pole-free summation turns a class of nonpolynomial finite sums into checked recurrences without losing the troublesome endpoints.

All three mechanisms have explicit input fragments, exact algorithms, compact certificate formats, and soundness arguments. Two have completeness arguments for the precisely stated unbounded fragments; the third has fixed-template coverage. Their Python prototypes and independent

replay code are available now. The remaining engineering work is sharply identified: source compilation, Lean replay, and the recurrence-comparison boundary. Completing those gates would add real capabilities to Forge without rebranding the concepts its repository already contains.

A Certificate schemas and executable entry points

A.1 Canonical scalar and polynomial encodings

A rational is encoded as the integer pair `[numerator, denominator]`, with positive denominator and greatest common divisor one. Python booleans are rejected even though they are subclasses of integers. Floats are rejected. A sparse polynomial in D variables is a strictly lexicographically ordered list

```
[[[e1, ..., eD], [numerator, denominator]], ...]
```

with nonnegative exponents, no repeated exponent vector, and no explicit zero coefficient. The empty list denotes the zero polynomial. Constant polynomials have the all-zero exponent vector. The source problem establishes the variable order; the certificate cannot reorder it.

The checker uses exponent-vector addition for monomial multiplication and collects equal exponents by exact addition. Substitution is evaluated by repeated squaring and cached powers. This implementation is independent of SymPy’s internal polynomial classes. Both search and checker still share the published mathematical schema, so common specification mistakes remain possible; the paper’s derivations and the concrete differential controls help expose them but do not formally exclude them.

A.2 Linear and ideal payloads

The linear problem supplies `dimension`, `actions`, `initial`, and `target`. Its certificate supplies `basis`, `target_weights`, and `action_weights`. Matrices are row-major. Every action must have one correctly dimensioned coefficient matrix.

The ideal problem additionally supplies `parameters`, with its initial coordinate polynomials in those parameter variables. Its certificate supplies `generators`, `target_weights`, and nested `action_weights`. The latter is indexed by action, destination generator, and source generator. Each entry is a polynomial in the state variables. There is no optional “trust this Gröbner basis” field.

A concrete counterexample payload contains a forward input `word` and its nonzero rational `value`; polynomial cases also contain concrete integer `parameters`. Replaying the original problem must reproduce that value. A nonzero value alone, without a valid word and initialization, is not accepted.

A.3 Summation payloads

The summation problem supplies `power` and a polynomial `weight` in k . The certificate supplies `p0`, `p1`, `r`, and `relation_multiplier`. Their variable orders are respectively (n) , (n) , (n, k) , and (A, B, n, k) . The local checker validates (13). Endpoints are justified by the fixed shape theorem, not by certificate-supplied numerical endpoint claims. An extension that changes this shape must provide a new boundary theorem.

A.4 Actual use from Python

The following is executable against the delivered package:

```
import sympy as s
from prototype.search import ideal_problem, ideal_search
from prototype.checkers import verify

x, y, z, w = s.symbols("x y z w")
a, b = s.symbols("a b")
problem = ideal_problem(
    (x, y, z, w),
    [(y*y, x, w*w, z)],
    (a, b),
    [a, b, a, -b],
    x - z,
)
result = ideal_search(problem, max_extensions=12)
assert result["status"] == "certified"
assert verify(problem, result["certificate"])
```

A user may replace the target or updates, but the certificate must then be rechecked against that changed problem. Reusing a previous result object or matching a case name does not establish validity.

B Reproduction and archive map

From the archive root:

```
python -m pip install -r requirements.txt
python -m prototype.run_experiments --output reproduced-results
python -S prototype/checkers.py reproduced-results/certificates \
    reproduced-results/counterexamples
python -m pytest -q
```

The regression tests replay the retained `results/` corpus; the generation command writes a distinct directory by default. Timings will vary across machines. The deterministic outcome categories and exact identities, not the millisecond values, are the reproducibility targets.

To build the article, run `python build.py`. It invokes `pdflatex` three times in a temporary output directory and copies the final PDF into `article/`. The complete source uses ordinary LaTeX packages; no font files are included. The generated `metrics.tex` and `recurrences.tex` reflect the retained run, not an automatically substituted local timing run.

The `lean/` directory is a separate, uncompiled specimen project. On a machine with the pinned Lean toolchain and fetched Mathlib dependencies, `lake build` is the intended first validation command. Its successful execution is *not* part of the recorded evidence supplied with this article.

Path	Contents
<code>article/</code>	Main TeX, section sources, generated tables, references, and rendered PDF.
<code>prototype/search.py</code>	The three untrusted exact search algorithms.
<code>prototype/checkers.py</code>	Standard-library proof and counterexample replay.

Path	Contents
<code>prototype/run_experiments.py</code>	Deterministic corpus, timing protocol, mutation generation, and exports.
<code>tests/</code>	Regression, schema, boundary, and differential tests.
<code>results/certificates/</code>	38 compact problem-and-certificate bundles.
<code>results/counterexamples/</code>	17 original-problem concrete counterexamples.
<code>results/summary.json</code>	Per-case outcomes, settings, timing statistics, and environment.
<code>results/mutations.json</code>	All 818 single-entry perturbation outcomes.
<code>results/*.log</code>	Accepted runs and explicitly separated development diagnostics.
<code>docs/</code>	Repository audit, validation scope, and source manifest.
<code>lean/</code>	Uncompiled semantic induction and worked replay specimens.

C Research questions left by the implementation

The most valuable unresolved questions are practical rather than foundational. How often is the target-generated ideal small in real Lean developments? When does fixed-degree affine lifting beat direct ideal closure? Which generator orders minimize kernel replay rather than search alone? Can the binomial certificate basis be enlarged while keeping endpoints polynomial? Which exact Leant candidate grammars admit useful observational congruences?

Each has a concrete experiment. Compare discovered generator counts under alternative term orders while replaying the same source goals. Measure monomial-lift dimensions and polynomial certificate sizes on matched affine problems. Compare proof construction and kernel time for expanded versus shared multiplier expressions. Add higher-shift binomial factors and require support and pole tests before admitting the new template. Restrict semantic deduplication to a source grammar with proved congruence lemmas and retain a no-pruning control.

None requires treating an unproved CAS assertion as evidence. The architecture can remain the one Forge already has; the advance comes from choosing mathematical fragments with finite, inspectable certificates.

References

- [1] Vladimir Reshetnikov, *Forge*, repository README, snapshot 674521027d968d59f7b83220ed52304a85cb55e2, reviewed 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/README.md>.
- [2] Forge, *Structural search*, merged article section, same snapshot. Especially polynomial recurrence synthesis and the state-machine generalization. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/05-structural.tex>.
- [3] Forge, *Implementation status*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/docs/STATUS.md>.
- [4] Forge, *Provenance: what came from where*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/article/sections/B-provenance.tex>.

- [5] Forge, *CertificateSpec and logical contracts*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/674521027d968d59f7b83220ed52304a85cb55e2/lean/Forge/Design/Contracts.lean>.
- [6] Vladimir Reshetnikov, *Leant*, README, snapshot 6bf05ad78c467989e68290f2d08bbbed40802d485, reviewed 15 September 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [7] Leant, *Behavioral assertions during synthesis*, same snapshot. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/behavioral-synthesis.md>.
- [8] Vladimir Reshetnikov, *Djex*, README, snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, reviewed 15 September 2026. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [9] Stefan Kiefer, *Notes on Equivalence and Minimization of Weighted Automata*, 2020, arXiv:2009.01217. In particular, the linear-space and equivalence algorithms. <https://arxiv.org/abs/2009.01217>.
- [10] David Cachera, Thomas Jensen, Arnaud Jobin, and Florent Kirchner, *Inference of Polynomial Invariants for Imperative Programs: A Farewell to Gröbner Bases*, Static Analysis (SAS 2012), Lecture Notes in Computer Science 7460. DOI: 10.1007/978-3-642-33125-1_7. https://doi.org/10.1007/978-3-642-33125-1_7. Related earlier report: *Fast inference of polynomial invariants for imperative programs*, INRIA RR-7627, 2011, HAL inria-00595783. The report’s HAL full text was access-blocked during preparation; no experimental claim here is borrowed from that inaccessible text.
- [11] Inria Celtique project team, *Fast inference of polynomial invariants*, 2014 activity report. Primary description of the team’s backward polynomial analysis and its implementation direction. <https://rex-radar.inria.fr/report/2014/celtique/uid36.html>.
- [12] Hadrien Brochet and Bruno Salvy, *Reduction-Based Creative Telescoping for Definite Summation of D-finite Functions*, arXiv:2307.07216, 2023, version 2. <https://arxiv.org/abs/2307.07216>.
- [13] SymPy development team, *Polynomials Manipulation Module Reference*, SymPy 1.14.0 documentation. Entries for `groebner`, `GroebnerBasis.reduce`, and exact polynomial domains. <https://docs.sympy.org/latest/modules/polys/reference.html>.
- [14] SymPy development team, *Concrete*, SymPy 1.14.0 documentation. Hypergeometric terms and `sympy.concrete.gosper.gosper_sum`. <https://docs.sympy.org/latest/modules/concrete.html>.
- [15] SageMath development team, *ore_algebra: Ore algebra*, Packages and Features reference, retrieved 15 September 2026. Optional package description and upstream implementation reference. https://doc.sagemath.org/html/en/reference/spkg/ore_algebra.html.
- [16] Mathlib contributors, *Multivariate polynomial evaluation*, Mathlib/Algebra/MvPolynomial/Eval.lean, revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Algebra/MvPolynomial/Eval.lean>.
- [17] Mathlib contributors, *Matrix multiplication*, Mathlib/Data/Matrix/Mul.lean, same revision. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Data/Matrix/Mul.lean>.
- [18] Mathlib contributors, *Binomial coefficients*, Mathlib/Data/Nat/Choose/Basic.lean, same revision. <https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Data/Nat/Choose/Basic.lean>.